Industrial Surface Defect Classification

Technical Summary **Team P03**

1. Problem and question

We classify cropped steel-surface defects into six NEU-CLS classes, using ImageNet-pretrained ResNet-18 with minimal preprocessing as the baseline. We ask whether the smaller EfficientNet-B0 preserves classification quality and whether camera-aware training augmentation improves robustness under controlled image degradation.

2. Dataset

NEU-CLS contains 1,800 grayscale 200×200 images: crazing, inclusion, patches, pitted surface, rolled-in scale, and scratches. Excluding one extra exact duplicate leaves 1,799 active images, split into 1,259 train, 180 validation, and 360 test crops, with 60 test images per class. Active paths, hashes, and reviewed duplicate groups do not cross split boundaries. All runs share this fixed split. Preprocessing replicates grayscale into three channels, resizes to 224×224 using antialiased bilinear interpolation, and applies ImageNet normalization.

3. Method and experimental setup

The controlled design compares ResNet-18/EfficientNet-B0 $\times$ minimal/camera-aware training $\times$ seeds 42/43/44, producing twelve checkpoints. Both architectures use ImageNet-1K V1 weights, a six-class head, and full fine-tuning. Camera-aware training adds small affine changes and exposure, contrast, gamma, noise, blur, or JPEG effects; evaluation preprocessing is deterministic. Shared training settings are AdamW, learning rate and weight decay 10^{-4}, batch 32, label smoothing 0.05, three warm-up epochs, cosine decay, a 50-epoch cap, and early-stopping patience 10. The earliest strict maximum of clean validation macro-F1 selects the checkpoint.

After checkpoints and analysis choices were frozen, all models were evaluated on clean test and nine corruption types at three severities: darkening, brightening, contrast, gamma, shot/read noise, Gaussian blur, horizontal/vertical motion blur, and JPEG compression. These 27 conditions provide a sensitivity analysis of the same 360 test crops.

We evaluated five clean macro-F1 contrasts: EfficientNet versus ResNet under each augmentation setting, camera-aware versus minimal training within each architecture, and the architecture–augmentation interaction. Each contrast used 5,000 paired, class-stratified bootstrap resamples, retaining 60 images per class and averaging differences across matched training seeds. The same sampled images were used for all models; checkpoints remained fixed.

4. Results

Configuration	Clean accuracy	Clean macro-F1	Mean corrupted macro-F1	T4 latency (ms/image)
ResNet-18 minimal	1.0000	1.0000 ± 0.0000	0.8293 ± 0.0077	3.65 ± 0.11
ResNet-18 camera	0.9963	0.9963 ± 0.0032	0.9577 ± 0.0152	3.60 ± 0.18
EfficientNet-B0 minimal	0.9963	0.9963 ± 0.0016	0.7387 ± 0.0346	8.85 ± 0.05
EfficientNet-B0 camera	0.9954	0.9954 ± 0.0058	0.9501 ± 0.0139	8.85 ± 0.10

Clean accuracy is the seed mean on the 360 original test images; clean macro-F1 is mean $\pm$ sample SD across seeds 42/43/44. Mean corrupted macro-F1 first averages the 27 condition-level scores within each seed, excluding clean, then reports mean $\pm$ sample SD across seeds. Macro-F1 weights all six classes equally. Latency is mean $\pm$ sample SD of three run-level medians; measurement details appear in Section 8.

All five clean-contrast 95% bootstrap intervals contain or touch zero. Separately, camera-aware training increases mean corrupted macro-F1 at every seed, with average gains of 0.1285 for ResNet and 0.2114 for EfficientNet. The bootstrap intervals concern clean macro-F1 only; they do not quantify uncertainty in the robustness gains.

5. Interpretation

EfficientNet achieves near-ceiling clean classification quality with about 64% fewer parameters, but the bootstrap analysis establishes neither a clear clean-performance advantage nor equivalence between configurations. Camera-aware training improves observed average corruption robustness for both architectures, although it does not improve every condition or clean probability quality. ResNet camera has the highest overall corrupted mean, but the camera-model ranking varies across seeds, and EfficientNet has higher batch-one T4 latency in this implementation.

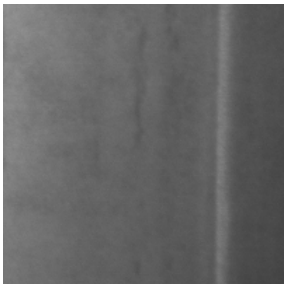
6. Limitations

The test contains 360 same-dataset crops with paired transformed views, rather than independent factory-camera images. NEU-CLS has no normal class, so localization and production false alarms are not evaluated. Corruption families overlap training augmentation, only three training seeds were used, and physical specimen identities are unavailable beyond reviewed duplicate groups; external acquisition data and component ablations are needed for broader claims.

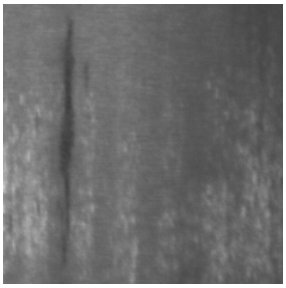
7. Failure analysis

The three images below are distinct, original clean-test crops. Confidence denotes the predicted-class probability. Proposed causes are visual hypotheses; dataset labels remain unchanged.

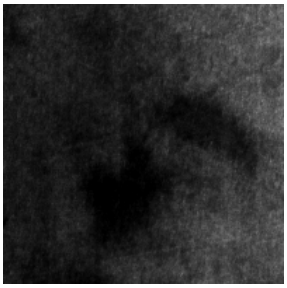
A. Sc_69.bmp



B. In_283.bmp



C. Pa_214.bmp



A. Scratches → inclusion. EfficientNet-B0 minimal, seed 44, confidence 0.7100. The faint, low-contrast elongated mark may resemble an inclusion. Six of twelve checkpoints make this confusion; all three minimal ResNets are correct.

B. Inclusion → scratches. ResNet-18 camera, seed 44, confidence 0.6134. Narrow vertical geometry may appear scratch-like. Only camera ResNet seeds 42 and 44 fail, showing seed-sensitive ambiguity.

C. Patches → pitted surface. EfficientNet-B0 camera, seed 42, confidence 0.3779 versus 0.3753 for patches. The mottled interior and weak boundary may resemble clustered pits. The other eleven checkpoints are correct.

8. Reproducibility

The **final Git repository** provides source code, configurations, setup/run instructions, environment records, and saved per-seed predictions and metrics. The **saved final Kaggle run** provides the experiment and downloadable weights. The **presentation slides** summarize the main findings, and a **recorded demo** is available.

Interactive demo. A local web application runs four trained seed-43 models: ResNet-18 and EfficientNet-B0, each with minimal or camera-aware training. Users can select prepared examples or upload a steel-defect image and compare predicted classes and probabilities side by side. All models run locally on CPU with shared preprocessing. The demo's Sc_69 confidence differs from the seed-44 example in the failure analysis because it uses seed 43.

Environment and runtime. Training and final evaluation used Python 3.13.15, PyTorch 2.11.0+cu128, torchvision 0.26.0+cu128, and CUDA 12.8 on a Tesla T4 in Kaggle. The local CPU demo was verified with Python 3.11.15. Training used mixed precision. Per-run training stages took approximately 41–65 seconds for ResNet and 68–97 seconds for EfficientNet, including validation and output generation; these times originate from different training sessions. Latency was measured using FP32, batch one, ten warm-up passes, and 50 synchronized timed forwards in a common T4 session, excluding loading, preprocessing, and CPU-to-GPU transfer. The CPU demo uses saved weights and requires no training.

9. Team contributions

- **Danil Khasanshin** (d.khasanshin@innopolis.university): coordinated the shared pipeline and Kaggle experiments, integrated the verified results, and assembled the repository code, README and reproducibility instructions.
- **Timur Kagarmanov** (t.kagarmanov@innopolis.university): independently audited the dataset and split, reproduced evaluation metrics and bootstrap results, checked representative failure cases, and worked on the technical summary.
- **Ruslan Nasibullin** (ru.nasibullin@innopolis.university): reviewed the six ResNet-18 runs, verified runtime measurements for all 12 checkpoints, and implemented and tested the offline inference demo.
- **Nikita Selezenev** (n.selezenev@innopolis.university): reviewed the six EfficientNet-B0 runs, compared robustness and calibration across all four configurations, and prepared comparison figures and interpretation of the results.